Walkthrough Video Script and Recording Kit

Company RAG Chatbot: final version. Follow the table row by row: left = what to do on screen, right = what to say.

How to use this document

• Target length: about 14 minutes 45 seconds (13:00 minimum if you speak faster; see 'If you must cut').

• Narration is written to be read aloud at a calm pace (about 140 words per minute). Say it in your own words; keep the numbers and technical terms.

• Text on a yellow background must be replaced with your own details (name, your questions, your MacBook numbers).

• Order of recording: record segments 0 to 9 in one session (they must stay in this order: segment 8 needs the documents that segment 9 deletes), then reset the stack and record segments 10 and 11 (testing and performance). Reason: the test scripts switch the AI settings to test models and need a stack without your real API key, so they must run last (details in 'Before you record').

Assignment coverage map

Assignment requirement	Where in the video
A. Core functionalities: what / how / technical logic	Segments 2 to 8 (each step explains the logic)
B. Backend/frontend interaction, data handling, logic	Segment 1 (architecture), segments 3, 5, 6 (narration), segment 8
C. Testing with different inputs, expected behaviour, error handling	Seg 2 (empty form, duplicate email), 3 (empty comment), 4 (wrong model), 5 (.exe, duplicate file), 6 (no evidence, 10th question), 9 (paused chat), 10 (invalid inputs)
D. At least two testing strategies: why, how, results, improvements	Segment 10 (unit, integration/E2E, security, load; bugs found and fixed)
E. Performance and compatibility (software and hardware)	Segment 11 (low-end VM vs MacBook, memory, browsers, phone layout)
Section 3 code quality: OOP, algorithms, structure	Segments 1 and 8
Section 1 scope alignment	Segment 12

Before you record

One week before

• Finish the three open items for the report: author details, screenshots, browser matrix (you will show browsers in segment 11).

• Run SuperAdmin > RAG Configuration > Languages > Run check once with your real Gemini key and note the numbers: you will show them in segment 4.

• Pick your demo documents: at least one English and one Kinyarwanda (or one bilingual document). Read them and prepare the five questions in the cheat sheet below.

• Decide your demo identities (names, emails, passwords) and write them in the cheat sheet. Use made-up emails.

Setup for segments 0 to 9 (real demo)

• Terminal in the project folder: docker compose down -v, then docker compose up --build -d. Wait for 'Application startup complete' in docker compose logs backend.

• In .env: EMBED_PAUSE_SECONDS=12 if you use Google's free tier. Do NOT open .env on screen (secrets).

• Log in as SuperAdmin, add your API key, set the models, Validate & activate, run the Language check. Then do nothing else: the company, employees and documents are created ON camera.

• Browser windows (all the same size, 125% zoom, bookmarks bar hidden): (1) Chrome normal = SuperAdmin. (2) Chrome Incognito = applicant, who becomes the company admin. (3) Safari or a second Chrome profile = employee. Log in only when the script says so.

• Create a dummy file for the rejection test: echo test > ~/Desktop/test.exe

• Pre-stage the chat limit (segment 6): the employee must already exist for this, so do it right after segment 5: in the employee window create one chat and ask 8 questions (any, e.g. repeat your Q1 with small changes). Leave that chat open in the sidebar. This saves two minutes on camera.

• Open these tabs/windows ready: README on GitHub (or in your editor), Technical_Report.pdf at Figure 1, VS Code with the project, Swagger (http://localhost:8000/docs).

• Mac: turn on Do Not Disturb, close Slack/Mail/WhatsApp, hide the Dock, plug in power, use headphones with a microphone or a quiet room.

Reset for segments 10 and 11 (testing and performance)

• Why: the test scripts switch the GLOBAL AI settings to test models and create test companies. smoke.py also checks that a configuration with no API key is rejected, so it needs a stack WITHOUT your key.

• Run: docker compose down -v, then docker compose up -d. Do not add an API key. Wait until http://localhost:8000/health says ok.

• In .env set REGISTER_LIMIT_PER_HOUR=1000 AND EMBED_PAUSE_SECONDS=0 (the benchmark ingests about 7,000 chunks; with a 12-second pause it would take about 30 minutes), then docker compose up -d backend.

• Prepare a Python environment once (before recording): cd backend && python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt numpy psutil

• In the same terminal: export DATABASE_URL=postgresql://rag_app:change_me_app@localhost:5432/company_rag, export SUPERADMIN_EMAIL=... and SUPERADMIN_PASSWORD=... (the ones in your .env).

• Dry-run once off camera: python tests/test_language.py, python tests/smoke.py, python tests/smoke_v11.py (about 1 minute in total on a Mac). All must say 'passed, 0 failed'. Then reset again (down -v, up -d) so the recording starts clean.

Recording settings

• macOS built-in: press Cmd+Shift+5 > Record Entire Screen > Options > Microphone: your mic > Save to: Desktop. (Or use OBS Studio.)

• Resolution: record at 1920 x 1080 or the Mac's native size; keep the browser window at 125% zoom so text is readable on a phone.

• Record each segment as a separate clip. A mistake then costs 1 minute, not 15. Name them 00_intro.mov, 01_architecture.mov and so on.

• Speak slowly. Pause 1 second before clicking a button. Move the mouse smoothly. Never type passwords in view (use fields that show dots).

• Never show: the API key, the .env file, real personal data.

Demo cheat sheet (fill in before recording)

SuperAdmin login	[email] / [password]
Applicant (becomes company admin)	[full name] / [phone] / [email] / [password]

Company name used in the request	[company name] (industry, services, city, employee count: [fill])
Employee	[full name] / [email] / [password]
Demo document(s)	[file name 1: language] [file name 2: language]
Q1 English question, answer is in the documents	[question] expected: [short answer] source file: [name]
Q2 Same question in Kinyarwanda	[question] expected: [short answer]
Q3 Cross-language: ask in one language, answer only exists in a document of the other	[question] expected: [short answer] source file: [name]
Q4 Unrelated question (nothing in the documents)	[question] expected: 'could not find' answer
Q5 Very short follow-up to Q1	[e.g. 'And for contractors?'] expected: [answer]
Numbers from your MacBook benchmark	login p50 [ms] chat p50 [ms] 10 users p50 [ms] / [req/s] ingestion [chunks/s] RAM idle -> load [MB]
Language check result (segment 4)	ask in Kinyarwanda, doc in English: match [x] vs unrelated [y]; ask in English, doc in Kinyarwanda: match [x] vs unrelated [y]

The script

Segment 0: Opening **0:00 - 0:45**

Covers: Context for the assessor (Section 1 and 2 introduction)

Do on screen	Say
Show the README top (title and table of links) or a title slide. Look at the camera if you wish.	Hello, I am [YOUR NAME], and this is the technical walkthrough of the final version of my capstone project: the Company RAG Chatbot. Employees lose a lot of time searching through company documents, and general chatbots do not know those documents and can invent answers. My platform lets every company upload its own documents. Employees then ask questions in English or Kinyarwanda and get answers that come only from those documents, with the sources shown.
Scroll to the 'Project overview' roles table in the README.	There are four kinds of users: visitors who request access for their company, the SuperAdmin who runs the platform, company admins who manage employees and documents, and employees who chat. In this video I will demonstrate the core features, explain the technical logic behind them, show my testing, my performance and compatibility results, and the code quality.

Segment 1: Architecture and technology — 0:45 - 2:00

Covers: B. Technical requirements: backend/frontend interaction, data handling, logic

Do on screen	Say
Show Figure 1 (architecture) in Technical_Report.pdf, or the diagram in the README.	The front end is Next.js 14 with React and TypeScript. The back end is FastAPI on Python 3.12. The database is PostgreSQL 16 with the pgvector extension, so documents, users and vectors live in one system. For AI I use Google's Gemini API: gemini-3.8-flash writes the answers and gemini-embedding-2 turns text into vectors. Everything runs in Docker.
Point at the arrows while you talk.	The browser talks to the back end only through a REST API with a signed token. Long jobs such as document processing, deletion and emails run in a background worker that takes jobs from a PostgreSQL queue, so uploads never block the application.
Switch to VS Code. Expand backend/app and show the folders routers, services, rag.	The code is layered. Routers only handle HTTP. Services hold the business logic. The rag folder holds the pipeline parts: text extraction, cleaning, chunking, language detection, retrieval and the AI providers. The front end follows the same idea with pages, reusable components and one typed API client.
Open db/DATABASE.md and show the ER diagram, or run the command on the right in a terminal. Terminal: docker compose exec db psql -U rag_owner -d company_rag -c '\dt'	The database has 33 tables. The RAG settings are stored as versioned JSON, every tenant record carries a company id, and PostgreSQL row-level security enforces company isolation even if application code had a bug.

Segment 2: Public site and company request — 2:00 - 3:00

Covers: A. Core functionality; C. Different inputs and error handling

Do on screen	Say
Applicant window (Incognito): open http://localhost:3000 and scroll slowly through the landing page.	This is the public site that a new company sees. Notice that all of this text comes from the database: the SuperAdmin can edit the sections, the onboarding pages and even the request form without changing code.
Click Request access. Create the account with the applicant details. Then click Sign out (bottom of the sidebar), click Request access again and register with the SAME email (test input). Show the error. Then go to Sign in and log in with the applicant account.	To request access, a company administrator first creates an account. If I sign out and try the same email again, the system refuses with a clear message instead of creating a duplicate. So I sign in with the account I just created.
On the request form, click Submit with the form EMPTY. Show the red field errors. Then fill it in using the cheat sheet and submit.	The request form is defined in the database and validated on the server. With empty fields I get field-level errors. Now I fill in the company details and submit. The request is now pending, and the applicant can have only one open request at a time; this is enforced by a database index, not just the screen.

Segment 3: Review workflow and onboarding — 3:00 - 4:15

Covers: A. Core functionality; privacy and terms requirement

Do on screen	Say
SuperAdmin window: open Company requests (note the badge), open the request. Click Request more info with the comment EMPTY.	The SuperAdmin sees the request in a review queue. I can approve, reject or ask for more information. Rejection and information requests require a comment, so I try with an empty comment and the system refuses.
Write: 'Please add your registration number.' Click Request more info. Switch to the applicant window, RELOAD the page, show the comment, type a reply, send.	Now the applicant sees my comment in the portal and also receives an email; in this local demo emails are only queued in an outbox table, because no mail server is configured. The applicant replies in the portal and the request goes back to pending.
SuperAdmin: click Approve. Applicant window: RELOAD the page, then click Start onboarding. Move through the three pages (guidelines, privacy summary, terms). Tick the terms checkbox and click Accept and open my workspace.	After approval the applicant goes through three pages: data guidelines, a privacy summary, and the terms and conditions. These pages are versioned in the database, so every acceptance is stored with the exact version, time and IP address. Only when the terms are accepted is the company created and the applicant becomes its administrator.
In the admin sidebar click Company request. Expand an accepted page.	From the sidebar the administrator can always come back to see the request history and exactly which terms were accepted.

Segment 4: Global RAG configuration — 4:15 - 5:15

Covers: A. Core functionality; C. Error handling; multilingual requirement

Do on screen	Say
SuperAdmin window: RAG Configuration. Click through the tabs Documents, Chunking, Embedding, Retrieval, LLM, Prompts quickly (2 seconds each).	The SuperAdmin controls the whole RAG pipeline from one screen: allowed file types and sizes, the chunking strategy including parent-child, the embedding and language models, retrieval settings, the prompts, and the error messages. Every part is separate, so any one of them can be replaced.
Open API credentials and point at the masked key.	API keys are encrypted in the database and are never sent back to the browser: only the last four characters are shown.
LLM tab: change the model name to gemini-wrong, click Validate and activate. Show the red error. Click Reset.	Now a safety feature. If I type a wrong model name, the system makes a real test call before activating anything. It fails, the new configuration is NOT activated, and the previous working one stays active for every company. Configurations are versioned and can be rolled back.
Languages tab: click Run check and show the result cards. Read the numbers from your cheat sheet.	Because the project must support Kinyarwanda, I built a Language check. It embeds a sentence in Kinyarwanda and its English translation and an unrelated sentence, and compares similarity. Asking in Kinyarwanda about an English document gives a match score of [x] against [y] for an unrelated one, and the other direction gives [x] against [y]. [Say honestly whether it is clearly better, and what the language model answered.]

Segment 5: Employees and knowledge base — 5:15 - 6:30

Covers: A. Core functionality; background processing; C. Different inputs

Do on screen	Say
Admin window: Employees > Add employee. Create the demo employee.	As company administrator I first add an employee. Accounts are limited to my company.
Knowledge base: drag test.exe into the upload area. Show the rejection message. Then upload your demo document(s).	Now the knowledge base. A file type that is not allowed, like this executable, is rejected with the reason. Now I upload my real document.
While it processes, point at the stage bar (extract, clean, chunk, embed, index) and the progress percent.	Processing runs in the background worker. It extracts the text, cleans it, splits it into chunks, creates embeddings and indexes them. I can watch each stage, and failures show a safe message with a Retry button.
Upload the SAME file again. Show the duplicate message. When ready, click EDA on the document and show the statistics and languages.	Uploading the same file again is detected through a content hash. When the document is ready, the analysis shows pages, words, removed headers and footers, and how many paragraphs are English or Kinyarwanda. Chunks never mix the two languages.
Dashboard: show the green status, click Start. Then do the chat pre-staging from the checklist (8 questions) if you have not yet.	The status turns green, and now I can Start the assistant. Pause would block usage but keep all data.

Segment 6: Employee chat — 6:30 - 8:15

Covers: A. Core functionality; C. Different inputs; algorithms in action

Do on screen	Say
Employee window: log in. Ask Q1 (English). Point at the answer and the source chip. Rate it 5 stars and answer True.	Now the employee experience. I ask [Q1]. The answer comes from my document and shows the source file. After the answer the system asks for feedback, a rating and a true or false question, which the SuperAdmin configured.
Ask Q2 (the same question in Kinyarwanda).	The same question in Kinyarwanda. The system detects the language and the answer comes back in Kinyarwanda.
Ask Q3 (cross-language).	Now the hard case. I ask in [language] about something that only exists in a document written in [other language]. Retrieval compares meaning, not words, so it still finds the right passage. [Say the result honestly.]
Ask Q4 (unrelated).	If I ask something that is not in the documents, the assistant does not invent an answer. It follows the no-evidence prompt and says it could not find it.
Ask Q5 (very short follow-up).	A very short follow-up like this still works, because the system adds the previous question to the search.
Open the pre-staged chat (8 of 9 used). Ask one more question, then try to type another. Show the banner.	Each chat is limited to nine exchanges, one question plus eight follow-ups. This limit is enforced in the back end and by a database constraint, not only on the screen. After the ninth, the chat asks me to start a new one.
Click Report. Choose a type, write a comment, submit.	Employees can also report an issue or send a recommendation, which the company administrator and the SuperAdmin can review.

Segment 7: Analytics, evidence and exports — 8:15 - 9:00

Covers: A. Core functionality; data handling

Do on screen	Say
Admin window: Analytics > Feedback. Show the row with the question, the chatbot answer and the score.	The administrator sees every rating together with the question and the exact answer the system gave.
Questions tab: show times asked and information found or not. Click Download CSV.	The questions report counts how often each question is asked and whether the documents contained the information, which reveals knowledge gaps. It can be downloaded as CSV.
Conversations tab: View a conversation. Point at 'Retrieved from' and the evidence badge.	For every answer the system stores which chunks were retrieved and their scores, so an administrator can investigate answer quality.

Segment 8: Code quality and algorithms — 9:00 - 10:15

Covers: Section 3: OOP, algorithms, code quality

Do on screen	Say
VS Code: open backend/app/rag/ai.py. Show GoogleEmbedder, FakeEmbedder, get_embedder; then GoogleLLM, FakeLLM, get_llm.	For object-oriented design: the embedding and LLM providers are classes with the same methods, created by factory functions. The rest of the code never depends on Google directly. In tests I swap in the fake classes, which proves they are interchangeable.
Open rag/retrieval.py and show rrf_fuse.	For algorithms: retrieval runs a vector search and a keyword search, then merges them with Reciprocal Rank Fusion, where each result scores one divided by sixty plus its rank. A result that both searches like rises to the top. Then the small matching chunk is replaced by its larger parent for context.
Open rag/chunking.py (language_segments, protect) and rag/lang.py (Detector).	Chunking keeps English and Kinyarwanda apart, protects abbreviations and decimals from false sentence breaks, and keeps headings with their text. Language detection compares the share of common words, with a confidence margin, and the language list lives in the database.
Open deps.py and show tenant() and role_user().	Access control is done through dependency injection. This function hands each endpoint the authenticated user and a database connection already scoped to that user's company.
Terminal: docker compose exec db psql -U rag_app -d company_rag -c 'select count(*) from chunks' then ... -c "begin; select set_config('app.bypass_rls','on',true); select count(*) from chunks; commit;"	And here is the row-level security: without a company context, the application's database role sees zero rows, even though the table has data. That is the last line of defence for company isolation.

Segment 9: Pause and end of service — 10:15 - 11:00

Covers: A. Core functionality; privacy and data lifecycle

Do on screen	Say
Admin dashboard: click Pause. In the employee window send a question and show the blocked message. Back in admin click Resume.	Pause blocks usage but preserves all data, so an administrator can control cost.
Click End service. Show the red warning. Click Download all my data, then type END and confirm. Show the deletion steps ticking off.	Ending the service is different. The administrator is advised to download all data first. Then deletion runs in six tracked steps: it keeps only anonymous rating statistics, then deletes vectors, database records including employee accounts, uploaded files, temporary files and logs. Each step is retried automatically if it fails.
Show the completed state and the empty knowledge base.	When it finishes, the company's data is gone and the administrator can start again by uploading new documents.

Segment 10: Testing strategies [record after the reset]	11:00 - 13:15
Covers: D. At least two testing strategies, with why / how / results / improvements; C. Different inputs	
Do on screen	**Say**
Terminal (venv active). Run: pytest tests/test_language.py tests/test_eval_metrics.py tests/test_security_and_pipeline.py -q. Wait for '39 passed'. Then show docs/testing/logs/02_unit_coverage_core_modules.log (86 percent).	I used six testing strategies. First, unit testing, chosen because the language, cleaning, ranking and security logic is pure and easy to get wrong. Thirty-nine tests check language detection, that chunks never mix languages, header and hyphen cleaning, Reciprocal Rank Fusion, password hashing, token expiry and tampering, and every evaluation formula against hand-computed values. All 39 pass, with 86 percent branch coverage on those modules.
Run: python tests/smoke.py then python tests/smoke_v11.py. Then python tests/smoke_eval.py and python tests/rest_interface_samples.py. Let the PASS lines scroll; end on the totals.	Second, functional and integration testing. Because the risk is in how the API, database, worker and AI adapters work together, these scripts drive the real HTTP API against PostgreSQL with deterministic test AI models. They cover whole journeys: login, configuration, upload, ingestion, chat, limits, analytics, export, deletion, the company request workflow and Kinyarwanda retrieval. 51 plus 76 checks, and all pass.
Scroll up to show a few specific PASS lines: 'company B cannot retrieve company A's documents', '10th question blocked by backend', 'accepting an outdated terms version is refused'.	Third, security and isolation testing: a multi-tenant system fails badly if data leaks. Tests try to read another company's conversations and documents, call admin endpoints with the wrong role, and I probed the database directly. All are refused.
Show the report page 'Testing' table or the list of bugs. (Optional: show the line 'PoolTimeout' in the old log, or the report text.)	The tests found real defects, which I fixed. A race where a new user could not log in right after being created, four failures out of thirty. Applicants were wrongly treated as belonging to an inactive company. And the load test found that with ten simultaneous users the database connection pool was exhausted and requests failed. Slow AI calls no longer hold a connection, and I added a regression test.
Open docs/testing/screenshots/12_superadmin_companies.png and the browser log docs/testing/logs/12_browser_e2e.log: '29 of 29 browser checks passed'.	Fourth, browser end-to-end testing with Playwright: a real Chromium signs in as each role, asks questions in English and Kinyarwanda, uploads documents, and checks that each role is kept out of the other areas. It also takes the screenshots in my report, and emulates an iPhone, a Pixel and an iPad. 29 of 29 pass.
SuperAdmin window: Model evaluation. Open the latest retrieval run: show Hit at 5, MRR, the dense versus keyword versus hybrid bars, then the LLM runs: oracle, pipeline and closed-book.	Fifth, evaluation of the AI itself. The Model evaluation page measures the embedding and retrieval part and the LLM separately on a benchmark in both languages, with baselines: keyword-only, embedding-only, and a model that sees no documents. [Read your REAL Gemini numbers here: hit at 5, MRR, key-fact recall, faithfulness.]
Total: 267 checks. Say the totals while showing the coverage line: 84 percent of the API code.	In total, 267 automated checks pass and the API code is 84 percent covered. I also tested with invalid inputs: wrong passwords, a sixth failed login, duplicate emails, a forbidden file type, a corrupt PDF, an outdated terms version, a paused company, and questions that have no answer. Sixth, acceptance testing with [N] participants gave an average SUS score of [x].

Segment 11: Performance and compatibility [record after the reset] 13:15 - 14:30

Covers: E. Performance and compatibility: software and hardware environments, resource usage

Do on screen	Say
Terminal: python tests/benchmark.py --part api. Let it run while you talk (about 40 to 60 seconds on a Mac). Show the printed numbers.	Now performance. This benchmark uses test AI models, so it measures my own system and not Google's response time, which in production is the biggest part of what a user waits for. On this MacBook: login takes [x] ms, which is deliberately slow because of password hashing. A chat question takes [x] ms at the median. Ten simultaneous users get [x] ms at the median, [x] requests per second and no errors.
Show the table in the technical report Section 5 (low-end VM versus MacBook).	I tested two hardware levels. A Linux test machine with two CPUs and eight gigabytes of memory, and this MacBook. On the test machine, ten simultaneous users still worked without errors, with a median of about 0.8 seconds and almost 10 requests per second; ingestion made about 450 chunks per second; and retrieval over twenty thousand chunks took about two milliseconds. In the browser, a page reaches its largest paint in about 1.2 seconds on an emulated mid-range phone, and about 7 seconds on a very slow 3G line.
Terminal: docker stats --no-stream. Point at the backend memory column.	For resource usage, the API process stays under 400 megabytes even during ingestion, and the two CPU cores saturate at ten simultaneous users, so capacity is CPU-bound.
Show the retrieval chart (Figure 2). Mention the honest finding.	One honest finding: at these sizes the vector index is no faster than an exact scan, because both take a couple of milliseconds. I keep the index for when the data grows. Its recall against exact search was 71 percent at 5,000 chunks and 89 percent at 20,000 on random vectors, which is a worst case.
Browsers: open the app in Chrome, Safari and Firefox (and Edge if you have it). In each: show the login page and a chat answer for 5 seconds. Then Chrome DevTools: Cmd+Option+I, Cmd+Shift+M, choose iPhone and show the layout.	For compatibility I tested the interface in Chrome, Safari and Firefox on macOS, and in the phone layout. [Say your results from the browser matrix, including any issue you found.] The front end is small: 87 kilobytes of shared JavaScript, 25 pages, and its load time under throttled phone profiles is in the testing report.

Segment 12: Scope, limitations and conclusion 14:30 - 15:15

Covers: Section 1: scope alignment; honest limitations

Do on screen	Say
Open docs/SCOPE_ALIGNMENT.md (or the PDF) and scroll the status column.	Against my approved scope, 30 of 36 features are completed, 4 are partial, and 2 are not implemented. The partial ones are the reranking model, the real server-driven progress indicator, the backup policy, which depends on the hosting provider, and acceptance testing with 20 or more participants, which I am completing now. Not implemented: [the cloud deployment status as in your report] and the future work items: billing, image understanding and marketing emails.
Show the 'Limitations and future work' section of the README, then the final slide or the README top again.	Known limitations: no OCR for scanned PDFs, answers are not streamed, email is outbound only, and Kinyarwanda quality depends on the Gemini models and needs review by a native speaker. In conclusion, the platform delivers a complete, tested, multi-tenant, bilingual RAG assistant. Thank you for watching. The code, README and technical report links are in the description.

If you must cut

- To save about 1:30: shorten segment 7 (analytics) to the CSV download and the evidence view only.
- To save about 1:00: in segment 4, skip clicking every tab; show only Embedding, Retrieval and the wrong-model test.
- To save about 1:00: in segment 8, show only ai.py and retrieval.py (keep the row-level-security command).

• Never cut: segment 6 (chat), the wrong-model test, the testing strategies, and the performance numbers: they carry the most rubric points.

After recording

• Edit: cut waiting time (document processing, slow answers), keep a cut visible with a short fade. Add simple captions if your English or Kinyarwanda audio is not clear.

• Export MP4 (H.264), 1080p. Attempt 2 needs the video file inside the ZIP, so keep the file under about 500 MB.

• Upload to YouTube as UNLISTED (or Google Drive with 'Anyone with the link can view'). Open the link in a private window to confirm it works without logging in.

• Paste the video link into README.md (top table) and docs/report/report_inputs.json, then run python docs/report/build_report.py again so the report shows it.

• Copy the chapter list below into the YouTube description.

YouTube chapters (paste into the description)

00:00 Introduction

00:45 Architecture and technology

02:00 Public site and company request

03:00 Review workflow and onboarding

04:15 Global RAG configuration and language check

05:15 Employees and knowledge base

06:30 Employee chat

08:15 Analytics and exports

09:00 Code quality and algorithms

10:15 Pause and end of service

11:00 Testing strategies

12:45 Performance and compatibility

14:00 Scope and conclusion